clients who are allowed to send traffic to the cache cluster, as shown next. You used the same approach in section 5.4:

```
Resources:
  # [...]
  ClientSecurityGroup:
    Type: 'AWS::EC2::SecurityGroup'
    Properties:
      GroupDescription: 'cache-client'
      VpcId: !Ref VPC
  CacheSecurityGroup:
    Type: 'AWS::EC2::SecurityGroup'
    Properties:
      GroupDescription: cache
      VpcId: !Ref VPC
      SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 6379
        ToPort: 6379
        SourceSecurityGroupId: !Ref ClientSecurityGroup     ①
```

① Only allows access from the ClientSecurityGroup

Attach the `ClientSecurityGroup` to all EC2 instances that need access to the cache cluster. This way, you allow access only to the EC2 instances that really need access.

Keep in mind that ElastiCache nodes always have private IP addresses. This means that you can't accidentally expose a Redis or Memcached cluster to the internet. You still want to use security groups to implement the principle of least privilege.

11.3.3 Controlling cluster and data access

Unfortunately, ElastiCache for Memcached does not provide user authentication. However, you have two different ways to authenticate users with ElastiCache for Redis:

- Basic token-based authentication
- Users with RBAC

Use token-based authentication when all clients are allowed to read and manipulate all the data stored in the cluster. Use RBAC if you need to restrict access for different users, for example, to make sure the frontend is only allowed to read some of the data and the backend is able to write and read all the data. Keep in mind that when